

Poradnik Pełnej Integracji: Circle Stack i Moduły Aplikacji

Ten poradnik jest bezpośrednią kontynuacją poprzednich i zakłada, że masz już wdrożone uwierzytelnianie, onboarding i podstawowe tworzenie portfeli.

Architektura Docelowa

Stworzymy i połączymy następujące moduły:

1. Backend (NestJS):

- `HealthModule` : Podstawowy endpoint statusu.
- `WebhooksModule` : Odbieranie powiadomień z Circle.
- `FiatModule` : Obsługa On-Ramp (wpłaty FIAT) i Off-Ramp (wyплаты FIAT).
- `GasModule` : Integracja z Circle Gas Station (Paymaster) do sponsorowania opłat.
- `CctpModule` : Obsługa transferów USDC między blockchainami (Cross-Chain).
- `FanModule` : Rozbudowa logiki dla fanów (historia, saldo).
- `SubscriptionModule` : Logika subskrypcji twórców.
- `AdminModule` : Załączek panelu administracyjnego.
- `NotificationModule` : System powiadomień wewnątrz aplikacji.
- Aktualizacja `PayoutsController` : Dodanie historii wypłat.

2. Frontend (Next.js):

- Nowe strony i komponenty w panelu fana i twórcy do obsługi nowych funkcji.
- Integracja z `apiClient` do komunikacji z nowymi endpointami.

Część 1: Rozbudowa Backendu (NestJS)

Zaczynamy od dodania wszystkich brakujących klocków po stronie serwera.

Krok 1: Aktualizacja Schematu `schema.prisma`

Dodajmy modele dla subskrypcji i powiadomień.

Code snippet

```
// prisma/schema.prisma

// ... (istniejące modele User, etc.)

model Subscription {
  id      String @id @default(cuid())
  fanId   String
  creatorId String
  tier     String // np. "basic", "premium"
  expiresAt DateTime
  isActive Boolean @default(true)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  fan     User @relation("FanSubscriptions", fields: [fanId], references: [id])
  creator User @relation("CreatorSubscriptions", fields: [creatorId], references: [id])

  @@unique([fanId, creatorId]) // Jeden fan może mieć jedną subskrypcję na twórcę
}

model Notification {
  id      String @id @default(cuid())
  userId  String // Do kogo jest powiadomienie
  type    String // np. "NEW_TIP", "SUBSCRIPTION_SUCCESS"
  message String
  isRead  Boolean @default(false)
  createdAt DateTime @default(now())

  user     User @relation(fields: [userId], references: [id])
}

// Zaktualizuj model User, dodając relacje
model User {
  // ... (istniejące pola)

  // NOWE RELACJE
  subscriptionsAsFan Subscription[] @relation("FanSubscriptions")
  subscriptionsAsCreator Subscription[] @relation("CreatorSubscriptions")
  notifications Notification[]
}
```

Po zmianach, uruchom migrację:

```
npx prisma migrate dev --name add_subscriptions_and_notificationsnpx prisma generate
```

Krok 2: Health Check (Standard)

To najprostszy, ale ważny element.

- `src/health/health.controller.ts` :TypeScript
- ```
import { Controller, Get } from '@nestjs/common';
```

```
@Controller('api/v1/health')
export class HealthController {
 @Get()
 check() {
 return {
 status: 'ok',
 timestamp: new Date().toISOString(),
 };
 }
}
```

- Dodaj `HealthController` do `AppModule` .

### Krok 3: Odbieranie Webhooków od Circle

Circle będzie nas informować o ważnych zdarzeniach (np. zakończony transfer). Musimy stworzyć endpoint, który będzie słuchał tych powiadomień.

- `src/webhooks/webhooks.controller.ts` :TypeScript

```
import { Controller, Post, Body, RawBodyRequest, Req } from '@nestjs/common';
import { WebhooksService } from './webhooks.service';

@Controller('api/v1/webhooks')
export class WebhooksController {
 constructor(private readonly webhooksService: WebhooksService) {}

 @Post('circle')
 handleCircleWebhook(@Req() req: RawBodyRequest<Request>, @Body() payload: any) {
 // Ważne: Circle wymaga weryfikacji sygnatury.
 // Użyj `rawBody` do weryfikacji, a `payload` do przetworzenia danych.
 const signature = req.headers['circle-signature'];
 const rawBody = req.rawBody; // Włącz `rawBody` w main.ts: `app.useBodyParser('json', { rawBody: true });`
 return this.webhooksService.processCircleWebhook(signature, rawBody, payload);
 }
}
```

- `src/webhooks/webhooks.service.ts` :TypeScript

```
import { Injectable } from '@nestjs/common';
import { NotificationService } from '../notification/notification.service'; // Stworzymy go

@Injectable()
export class WebhooksService {
 constructor(private notificationService: NotificationService) {}

 async processCircleWebhook(signature: string, rawBody: Buffer, payload: any) {
 // KROK 1: Weryfikacja sygnatury (PSEUDOKOD - zaimplementuj zgodnie z dokumentacją Circle)
 const isValid = this.verifySignature(signature, rawBody);
 if (!isValid) {
 // Zwróć błąd lub zignoruj
 return;
 }

 // KROK 2: Przetwarzanie zdarzenia
 }
```

```

switch (payload.notificationType) {
 case 'wallets.settlements.completed':
 // Użytkownik zasilił konto przez On-Ramp
 // TODO: Zaktualizuj saldo użytkownika w swojej bazie
 await this.notificationService.create(payload.wallet.userId, 'DEPOSIT_SUCCESS', 'Your deposit has been
completed.');
```

```

 break;
 case 'transfers.completed':
 // Transfer został zakończony
 // TODO: Zaktualizuj stan transakcji w swojej bazie
 break;
 // ... inne typy powiadomień
 }

 return { received: true };
}

private verifySignature(signature: string, body: Buffer): boolean {
 // TODO: Zaimplementuj logikę weryfikacji sygnatury z dokumentacji Circle
 // Będziesz potrzebować swojego sekretne klucza z Circle.
 return true; // Placeholder
}
}

```

## Krok 4: Circle Paymaster (Gas Station)

Sponsorowanie opłat transakcyjnych jest kluczowe dla dobrego UX.

- `src/gas/gas.service.ts` :TypeScript

```

import { Injectable } from '@nestjs/common';
import { CircleService } from '../circle/circle.service'; // Zakładając, że masz centralny serwis Circle

@Injectable()
export class GasService {
 constructor(private circleService: CircleService) {}

 // Ta funkcja będzie używana przez inne serwisy (np. TipsService)
 async sponsorTransaction(userId: string, transactionDetails: any) {
 const circleClient = this.circleService.getClient(); // Pobierz klienta SDK

 // Logika wysłania transakcji przez Gas Station Circle
 // SDK Circle powinno mieć metody do tworzenia i wysyłania transakcji
 // z użyciem Paymastera.
 const response = await circleClient.createTransaction({
 // ... szczegóły transakcji
 feeLevel: 'HIGH', // Pozwala Circle zarządzać opłatą
 walletId: this.circleService.getUserWalletId(userId),
 });

 // Monitoruj status tej transakcji (np. przez webhooks)
 return response.data;
 }
}

```

**Jak to działa?** Zamiast wysyłać transakcję bezpośrednio, zlecasz ją do Circle z prośbą o opłacenie gazu. W `TipsService` czy `SubscriptionService`, zamiast `wallet.sendTransaction(...)` użyjesz `gasService.sponsorTransaction(...)`.

## Krok 5: Circle CCTP (Cross-Chain Transfers)

Umożliwienie użytkownikom przesyłania USDC między sieciami (np. z Polygon na Ethereum).

- `src/cctp/cctp.controller.ts`: TypeScript

```
import { Controller, Post, Body, UseGuards, Request } from '@nestjs/common';
import { JwtAuthGuard } from '../auth/guards/jwt-auth.guard';
import { CctpService } from './cctp.service';

@UseGuards(JwtAuthGuard)
@Controller('api/v1/cctp')
export class CctpController {
 constructor(private cctpService: CctpService) {}

 @Post('initiate')
 initiateTransfer(@Request() req, @Body() body: { amount: string; destinationChain: string }) {
 return this.cctpService.initiateCrossChainTransfer(req.user.id, body.amount, body.destinationChain);
 }
}
```

- `src/cctp/cctp.service.ts`: TypeScript

```
import { Injectable } from '@nestjs/common';
import { CircleService } from '../circle/circle.service';

@Injectable()
export class CctpService {
 constructor(private circleService: CircleService) {}

 async initiateCrossChainTransfer(userId: string, amount: string, destinationChain: string) {
 // KROK 1: Spalenie tokenów na sieci źródłowej (burn)
 // To jest zazwyczaj transakcja on-chain, którą musisz zainicjować
 const burnTx = await this.circleService.initiateBurn(userId, amount);

 // KROK 2: Oczekiwanie na finalizację i pobranie atestacji od Circle
 // Może to zająć kilka minut. Możesz użyć webhooków lub pollingu.
 const attestation = await this.circleService.getAttestationForBurn(burnTx.hash);

 // KROK 3: Wybicie (mint) tokenów na sieci docelowej, używając atestacji
 const mintTx = await this.circleService.mintOnDestination(attestation, destinationChain);

 return { success: true, mintTxHash: mintTx.hash };
 }
}
```

**Uwaga:** To jest uproszczony schemat. Prawdziwa implementacja wymaga starannego zarządzania stanem transakcji, obsługi błędów i

prawdopodobnie systemu kolejkowego (np. Redis/BullMQ) do monitorowania długotrwałych operacji.

## Krok 6: On-Ramp / Off-Ramp (Wpłaty i Wyплаты)

- `src/flat/flat.controller.ts` :TypeScript

```
import { Controller, Post, Body, UseGuards, Request, Get } from '@nestjs/common';
import { JwtAuthGuard } from '../auth/guards/jwt-auth.guard';
import { FiatService } from './flat.service';

@UseGuards(JwtAuthGuard)
@Controller('api/v1/flat')
export class FiatController {
 constructor(private fiatService: FiatService) {}

 @Post('deposit/initiate')
 initiateDeposit(@Request() req, @Body() body: { amount: string; method: string }) {
 // method może być 'credit-card', 'bank-transfer' etc.
 return this.fiatService.createDeposit(req.user.id, body.amount, body.method);
 }

 @Post('withdraw/initiate')
 initiateWithdrawal(@Request() req, @Body() body: { amount: string; destinationAccountId: string }) {
 // destinationAccountId to ID konta bankowego w Circle
 return this.fiatService.createWithdrawal(req.user.id, body.amount, body.destinationAccountId);
 }
}
```

- `src/flat/flat.service.ts` :TypeScript

```
import { Injectable } from '@nestjs/common';
import { CircleService } from '../circle/circle.service';

@Injectable()
export class FiatService {
 constructor(private circleService: CircleService) {}

 async createDeposit(userId: string, amount: string, method: string) {
 // Użyj API Circle Payments do stworzenia płatności
 // To zwróci URL do strony płatności lub inne instrukcje dla użytkownika
 const paymentIntent = await this.circleService.getClient().createPayment({
 amount: { amount, currency: 'USD' },
 // ... inne szczegóły
 });
 return paymentIntent;
 }

 async createWithdrawal(userId: string, amount: string, destinationAccountId: string) {
 // Użyj API Circle Payouts do stworzenia wypłaty
 const payout = await this.circleService.getClient().createPayout({
 amount: { amount, currency: 'USD' },
 destination: { type: 'wire', id: destinationAccountId },
 });
 return payout;
 }
}
```

```
}
}
```

## Krok 7: Subskrypcje i Panel Fana

- `src/subscriptions/subscriptions.controller.ts` :TypeScript

```
// ...
@UseGuards(JwtAuthGuard)
@Controller('api/v1/subscriptions')
export class SubscriptionsController {
 // POST / - zasubskrybuj twórcę
 // GET /creator/:id - pobierz subskrybentów twórcy
 // GET /fan/:id - pobierz subskrypcje fana
}
```

- `src/fan/fan.controller.ts` :TypeScript

```
// ...
@UseGuards(JwtAuthGuard)
@Controller('api/v1/fan')
export class FanController {
 // GET /wallet/balance - saldo (już masz)
 // GET /wallet/transactions - historia transakcji
 // GET /subscriptions - lista subskrybowanych twórców
}
```

---

## Część 2: Implementacja we Frontendzie (Next.js)

Teraz, gdy backend ma wszystkie potrzebne endpointy, możemy zbudować interfejs użytkownika.

## Krok 8: Panel Fana

Stwórz nową sekcję w aplikacji pod `app/(dashboard)/fan/`.

- `app/(dashboard)/fan/wallet/page.tsx` :
  - Komponent wyświetlający saldo (użyj hooka `useBalance`).
  - Komponent do inicjowania wpłat (On-Ramp) i wypłat (Off-Ramp). Będzie on renderował formularz i wywoływał `apiClient.post('/fiat/deposit/initiate', ...)`.
  - Komponent historii transakcji, który pobiera dane z `/api/v1/fan/wallet/transactions`.
- `app/(dashboard)/fan/subscriptions/page.tsx` :
  - Lista twórców, których fan subskrybuje, pobrana z `/api/v1/fan/subscriptions`.

## Krok 9: Komponenty dla CCTP i Subskrypcji

- **Komponent CCTP:**
  - Formularz z polem na kwotę i wyborem sieci docelowej.
  - Po kliknięciu "Transferuj", wywołuje `apiClient.post('/cctp/initiate', ...)`.
  - Wyświetla status operacji (np. "Krok 1/3: Palenie tokenów...").
- **Przycisk "Subskrybuj" na profilu twórcy:**
  - Po kliknięciu, otwiera modal z wyborem progu subskrypcji.
  - Wywołuje `apiClient.post('/subscriptions', { creatorId, tier })`.
  - Transakcja opłaty za subskrypcję powinna być sponsorowana przez **Gas Station**, więc backend `SubscriptionsService` powinien użyć `GasService`.

## Krok 10: Wyświetlanie Powiadomień

- Stwórz globalny hook, np. `useNotifications`, który cyklicznie (co 15-30 sekund) odpytuje endpoint `/api/v1/notifications`.
- Stwórz komponent "dzwoneczka" w głównym layoutcie, który wyświetla liczbę nieprzeczytanych powiadomień.
- Po kliknięciu, pokazuje listę ostatnich powiadomień.

## Podsumowanie Pełnego Przepływu

1. **Rejestracja i Onboarding:** Bez zmian, ale po `choose-username` użytkownik łąduje w swoim panelu.
2. **Zasilenie konta (On-Ramp):** Fan w swoim panelu klika "Wpłać", podaje kwotę. Frontend przekierowuje go do interfejsu płatności Circle. Po udanej płatności, **webhook** z Circle informuje nasz backend, który aktualizuje saldo użytkownika.
3. **Subskrypcja:** Fan na profilu twórcy klika "Subskrybuj". Backend tworzy subskrypcję i zleca transakcję opłaty. **Gas Service** sponsoruje opłatę.
4. **Transfer Cross-Chain (CCTP):** Twórca chce przenieść zarobione USDC z Polygon na Ethereum. Używa interfejsu CCTP w swoim panelu. Backend zarządza procesem burn-and-mint.
5. **Wypłata (Off-Ramp):** Twórca chce wypłacić środki na swoje konto bankowe. Inicjuje wypłatę w panelu. Backend używa API Payouts Circle.



6. **Powiadomienia:** O wszystkich kluczowych zdarzeniach (nowy tip, udana wypłata) użytkownik jest informowany przez system powiadomień w aplikacji.

Ta architektura tworzy kompletny, spójny i potężny system, w pełni wykorzystujący możliwości Circle i zapewniający świetne doświadczenie zarówno dla fanów, jak i twórców.